KNN, DECISION TREE, LOGISTIC REGRESSION

MACHINE LEARNING

Dosen Pengampu :

Ramadhana Setiyawan., M.Cs.

Dibuat oleh:

Eryy Agus Muklis

NIM. 25050530118

PENDIDIKAN TEKNIK INFORMATIKA

UNIVERSITAS NEGERI YOGYAKARTA

TAHUN 2026

1. Logistic Regression
   a. Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm

from statsmodels.stats.outliers_influence import
variance_inflation_factor # untuk mengalkulasi nilai VIF

from sklearn.model_selection import train_test_split # untuk membagi
data secara acak
from sklearn.metrics import accuracy_score # untuk menghitung nilai
akurasi

import warnings
warnings.filterwarnings("ignore") # untuk menghilangkan warning yang
muncul setelah menjalankan cell
```

   b. Load Dataset

```
df = sns.load_dataset('titanic')
df
```

Hasil setelah di run :

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
886	0	2	male	27.0	0	0	13.0000	S	Second	man	True	NaN	Southampton	no	True
887	1	1	female	19.0	0	0	30.0000	S	First	woman	False	B	Southampton	yes	True
888	0	3	female	NaN	1	2	23.4500	S	Third	woman	False	NaN	Southampton	no	False
889	1	1	male	26.0	0	0	30.0000	C	First	man	True	C	Cherbourg	yes	True
890	0	3	male	32.0	0	0	7.7500	Q	Third	man	True	NaN	Queenstown	no	True

891 rows × 15 columns

Next steps: Generate code with df | New interactive sheet

```
df[['survived', 'pclass', 'sex', 'age', 'fare']]
```

Hasil setelah data yang ditampilkan hanya data tertentu:

	survived	pclass	sex	age	fare
0	0	3	male	22.0	7.2500
1	1	1	female	38.0	71.2833
2	1	3	female	26.0	7.9250
3	1	1	female	35.0	53.1000
4	0	3	male	35.0	8.0500
...	...	...	...	...	...
886	0	2	male	27.0	13.0000
887	1	1	female	19.0	30.0000
888	0	3	female	NaN	23.4500
889	1	1	male	26.0	30.0000
890	0	3	male	32.0	7.7500

891 rows × 5 columns

c. Define X and y

Memisahkan data x dan y

```
X = df[['pclass', 'sex', 'age', 'fare']]
y = df['survived']
```

d. Imputing missing values

```
# Check missing values
X.isna().sum()
```

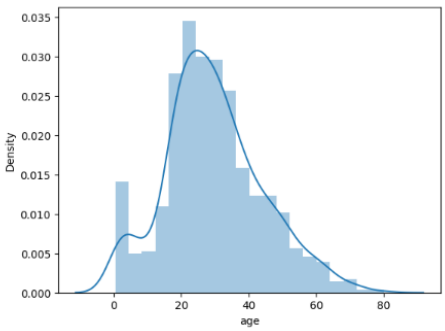
Hasil setelah di run :

	0
pclass	0
sex	0
age	177
fare	0

dtype: int64

```
# Ada missing value di feature 'age'
sns.distplot(df['age']);

# Distribusi tidak normal --> impute missing value pakai
median
```

Hasilnya :




```
# Normality test check for imputing strategy
from scipy.stats import normaltest
normaltest(df['age'].dropna())
```

Hasilnya :

```
NormaltestResult(statistic=np.float64(18.105032952089758),
pvalue=np.float64(0.00011709599657350757))
```

```
# Check the median of 'age'
X['age'].median()
```

Hasil dari median X 'age':

```
28.0
```

```
# Fill the missing values
X['age'] = X['age'].fillna(X['age'].median())
```

```
# Recheck missing value
X.isna().sum()
```

Hasilnya :

	0
pclass	0
sex	0
age	0
fare	0

dtype: int64

e. Create a dummy variable

```
X = pd.get_dummies(X, columns=['sex'], drop_first=True)
X
```

Hasilnya :

	pclass	age	fare	sex_male
0	3	22.0	7.2500	True
1	1	38.0	71.2833	False
2	3	26.0	7.9250	False
3	1	35.0	53.1000	False
4	3	35.0	8.0500	True
...	...	...	...	...
886	2	27.0	13.0000	True
887	1	19.0	30.0000	False
888	3	28.0	23.4500	False
889	1	26.0	30.0000	True
890	3	32.0	7.7500	True

891 rows × 4 columns

xt steps: Generate code with X | New interactive sheet

f. Multicollinearity check

```
# Function to calculate VIF
def calc_vif(X):
    vif = pd.DataFrame()
    vif['variables'] = X.columns
    vif['VIF'] = [variance_inflation_factor(X.values, i) for
i in range(X.shape[1])]
    vif['Acceptable'] = np.where(vif.VIF < 4, 'Yes', 'No')
    return (vif)
```

```
X_numerical = X.astype(int)
calc_vif(X_numerical)
```

Hasilnya :

	variables	VIF	Acceptable
0	pclass	3.729256	Yes
1	age	4.049073	No
2	fare	1.418839	Yes
3	sex_male	2.900925	Yes

g. Data Splitting

```
# Splitting data
X_train, X_test, y_train, y_test= train_test_split(
    X,
    y,
    stratify = y, # digunakan untuk menjaga proporsi tiap
kelas sesuai dengan data keseluruhan
    test_size = 0.2, # ukuran test set --> mengambil 20%
data dari dataset awal
    random_state = 42 # digunakan agar output yang
dihasilkan tetap sama meskipun di-run berulang kali
)
```

```
df['survived'].value_counts()
```

Hasilnya :

survived	count
0	549
1	342

dtype: int64

```
# Check the shapes
print(X.shape)           # Jumlah total (100%)
print(X_train.shape)     # 80% dari total data
print(X_test.shape)      # 20% dari total data
```

```
Hasilnya :
(891, 4)
(712, 4)
(179, 4)
```

h. Logistic regression modelling using statsmodels

```
# model
sm_logit = sm.Logit(y_train,
sm.add_constant(X_train.astype(float)))

# training
result = sm_logit.fit()
```

Hasilnya :

```
Optimization terminated successfully.
         Current function value: 0.445883
         Iterations 6
```

```
print(result.summary())
```

Hasilnya:

```
                           Logit Regression Results
==============================================================================
Dep. Variable:                survived   No. Observations:                  712
Model:                           Logit   Df Residuals:                      707
Method:                            MLE   Df Model:                            4
Date:                 Wed, 29 Apr 2026   Pseudo R-squ.:                  0.3302
Time:                         04:18:13   Log-Likelihood:                -317.47
converged:                       True   LL-Null:                       -473.99
Covariance Type:            nonrobust   LLR p-value:                 1.662e-66
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const          4.7196      0.575      8.210      0.000       3.593       5.846
pclass        -1.1666      0.154     -7.568      0.000      -1.469      -0.864
age           -0.0347      0.008     -4.187      0.000      -0.051      -0.018
fare           0.0010      0.002      0.397      0.691      -0.004       0.006
sex_male      -2.6293      0.211    -12.445      0.000      -3.043      -2.215
==============================================================================
```

i. Interpretasi

```
# Interpretasi hanya berlaku/valid pada rentang berikut
df.describe().loc[['min', 'max']][['pclass', 'age', 'fare']]
```

```
# Interpretasi pclass

c = 3
a = 1 # pilih c dan a pada rentang 1-3, dengan c > a
Beta = -1.1666

OR_pclass = np.exp(Beta*(c-a))
print('OR_pclass =', OR_pclass)

OR_pclass_interpretation = 1/OR_pclass
print('OR_pclass_interpretation =',
OR_pclass_interpretation)
```

Hasil setelah di run :

```
OR_pclass = 0.09698489832213165
OR_pclass_interpretation = 10.310883625186037
```

```
# Interpretasi age
c = 50
a = 40 # pilih c dan a pada rentang 0-80, dengan c > a
Beta = -0.0347

OR_age = np.exp(Beta*(c-a))
print('OR_age =', OR_age)
OR_age_interpretation = 1/OR_age
print('OR_age_interpretation =', OR_age_interpretation)
```

```
Hasil setelah di run:
OR_age = 0.7068053282577494
OR_age_interpretation = 1.4148167253704287
```

```
# Interpretasi sex_male
c = 1
a = 0
Beta = -2.6293
OR_sex_male = np.exp(Beta*(c-a))
print('OR_sex_male =', OR_sex_male)
OR_sex_male_interpretation = 1/OR_sex_male
print('OR_sex_male_interpretation =',
OR_sex_male_interpretation)
```

```
Hasilnya:
OR_sex_male = 0.07212893482567767
OR_sex_male_interpretation = 13.864061661479065
```

j. Validate the model using test set

```
# Check the model result
y_predict_proba =
result.predict(sm.add_constant(X_test.astype(float)))
y_predict_class = np.where(y_predict_proba > .5, 1, 0)
```

```
y_predict_proba
```

Hasilnya :

	0
565	0.098200
160	0.051245
553	0.102959
860	0.056438
241	0.565801
...	...
880	0.824285
91	0.109600
883	0.230954
473	0.832426
637	0.215622

179 rows × 1 columns

dtype: float64

```
y_predict_class
```

Hasilnya :

```
array([0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1,
       0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1,
       1, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1,
       1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 1,
       0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0,
       1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1,
       0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0,
       0, 1, 0])
```

k. Evaluation metrics

```
print('Model accuracy score in the test set:',
accuracy_score(y_test, y_predict_class))
```

Hasil setelah di run:

```
Model accuracy score in the test set: 0.7932960893854749
```

Interpretasi hasil metrics

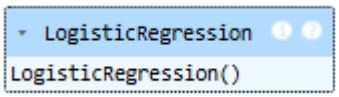
Katakanlah ada 100 penumpang yang diprediksi oleh model, maka 79 penumpang akan terprediksi (survived or not survived) dengan benar.

l. Check the accuracy of model with sklearn

```
from sklearn.linear_model import LogisticRegression
```

```
# define model
model = LogisticRegression()

# fitting
model.fit(X_train, y_train)
```

Hasilnya setelah di run :

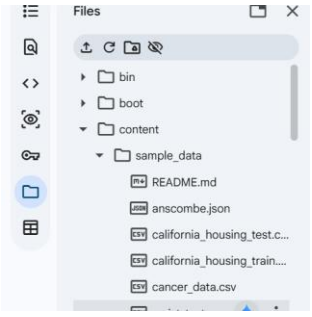



```
# predict
y_pred = model.predict(X_test)

# accuracy
accuracy_score(y_test, y_pred)
```

```
Hasilnya setelah di run:

0.7877094972067039
```

2. KNN & Decision Tree
   a. Import dataset

   Untuk dataset cancer.csv yang ada kita upload pada google colap ke dalam /content/sample_data supaya data bisa kita gunakan.




```
# df = pd.read_csv('cancer_data.csv')
df = pd.read_csv('/content/sample_data/cancer_data.csv')
df.head()
```

Hasil Import datanya :

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430

5 rows × 33 columns

- diagnosis:
  M: malignant (ganas)
  B: benign (jinak)
- concave points (number of concave portions of the contour)
- texture (standard deviation of gray-scale values)

```
# Ubah category M (malignant) jadi 1 dan B (benign) jadi 0
df['diagnosis'] = df['diagnosis'].apply(lambda x: 1 if x == 'M' else 0)
```

```
df['diagnosis']
```

Hasilnya setelah di run:

	diagnosis
0	1
1	1
2	1
3	1
4	1
...	...
564	1
565	1
566	1
567	1
568	0

569 rows × 1 columns

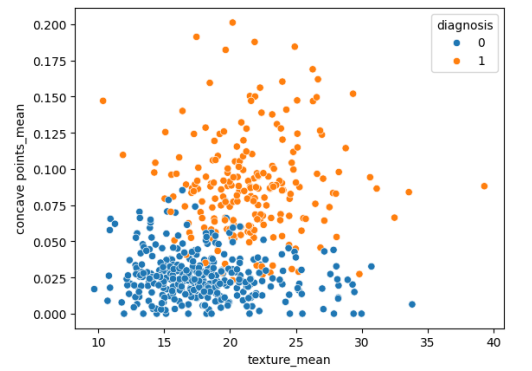
dtype: int64

```
# Features dan Target
features = ['texture_mean', 'concave points_mean']
target = ['diagnosis']
X = df[features]
y = df[target]
```

b. EDA : Exploratory Data Analysis

```
# Berdasarkan 2 features: 'texture_mean' dan 'concave points_mean', terlihat cancer yang M dan B cukup terpisah
sns.scatterplot(x = 'texture_mean', y = 'concave points_mean', data = df, hue = 'diagnosis');
```

Hasilnya setelah di klik run :




```
# Check missing value
df[['texture_mean', 'concave points_mean']].isnull().sum()
```

Hasil setelah di klik run:

	0
texture_mean	0
concave points_mean	0

dtype: int64

c. Data Splitting

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    stratify = y,
    test_size = 0.2,
    random_state = 42)
```

d. KNN

a) Without scaling

```
# define model
knn = KNeighborsClassifier(n_neighbors=3)

# training model (KNN belajar dari training set)
knn.fit(X_train, y_train)

# predict (KNN memprediksi M atau B berdasarkan features di test set )
y_predict_class = knn.predict(X_test)
```

```
print('Model accuracy in test dataset:',
accuracy_score(y_test, y_predict_class))
```

Hasilnya :

```
Model accuracy in test dataset: 0.8859649122807017
```

b) With scaling

```
scaler = MinMaxScaler()
```

```
scaler.fit(X_train) # preprocess fit, diaplikasikan
hanya pada data training

X_train_scaled = scaler.transform(X_train) # transform
data X_train
X_test_scaled = scaler.transform(X_test) # transform
data x_test
```

```
# define model
knn = KNeighborsClassifier(n_neighbors=3)

# training model
knn.fit(X_train_scaled, y_train) # model fit

# predict
y_predict_class = knn.predict(X_test_scaled) # model
predict
```

```
print('Model accuracy in test dataset:',
accuracy_score(y_test, y_predict_class))
```

Hasilnya :

```
Model accuracy in test dataset: 0.9210526315789473
```

Hasil akurasi dengan metode KNN dengan menggunakan hanya menggunakan 2 features dan scaling sebesar 92.1%

c) With StandardScaler

```
# Initialize StandardScaler
scaler_standard = StandardScaler()

# Fit on training data and transform both training and
test data
X_train_standard_scaled =
scaler_standard.fit_transform(X_train)
X_test_standard_scaled =
scaler_standard.transform(X_test)
```

```
# define model with best k from previous analysis (or a
default, e.g., 9)
knn_standard = KNeighborsClassifier(n_neighbors=9) #
Using 9 as an example best_k

# training model
knn_standard.fit(X_train_standard_scaled, y_train)

# predict
y_predict_class_standard =
knn_standard.predict(X_test_standard_scaled)
```

```
print('Model accuracy in test dataset with
StandardScaler:', accuracy_score(y_test,
y_predict_class_standard))
```

Hasilnya :

```
Model accuracy in test dataset with StandardScaler:
0.9385964912280702
```

d) The best K Factor

```
# mencari kisaran range untuk menentukan K factor
terbaik dengan mengakarkan jumlah data
np.sqrt(df.shape[0])
```

Hasilnya :

```
np.float64(23.853720883753127)
```

```
# Find the best k

k = range(1, 30, 2)
testing_accuracies = []
training_accuracies = []
score = 0

for i in k:

    # model
    knn = KNeighborsClassifier(n_neighbors = i)

    # training scaled data
    knn.fit(X_train_scaled, y_train)

    # akurasi pada training set
    y_predict_train = knn.predict(X_train_scaled)
    training_accuracies.append(accuracy_score(y_train,
y_predict_train))

    # akurasi pada test set
    y_predict_test = knn.predict(X_test_scaled)
    acc_score = accuracy_score(y_test, y_predict_test)
    testing_accuracies.append(acc_score)

    # jika ada akurasi yg lebih baik,
    if score < acc_score:
        score = acc_score
        best_k = i
```

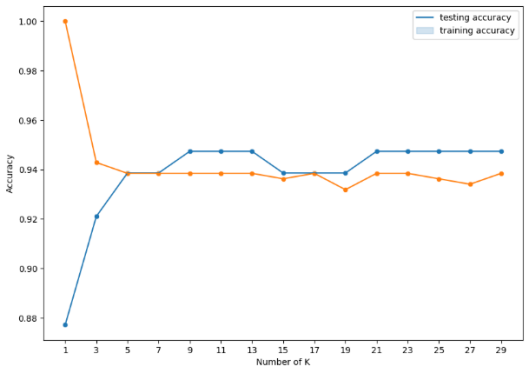
```
testing_accuracies
```

Hasil setelah di klik run:

```
[0.8771929824561403,
 0.9210526315789473,
 0.9385964912280702,
 0.9385964912280702,
 0.9473684210526315,
 0.9473684210526315,
 0.9473684210526315,
 0.9385964912280702,
 0.9385964912280702,
 0.9385964912280702,
 0.9473684210526315,
 0.9473684210526315,
 0.9473684210526315,
 0.9473684210526315,
 0.9473684210526315]
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# Plot the accuracies result
plt.figure(figsize=(10, 7))

# akurasi pada test set
sns.lineplot(x=k, y=testing_accuracies)
sns.scatterplot(x=k, y=testing_accuracies)

# akurasi pada train set
sns.lineplot(x=k, y=training_accuracies)
sns.scatterplot(x=k, y=training_accuracies)

plt.legend(['testing accuracy', 'training accuracy'])
plt.xlabel('Number of K')
plt.ylabel('Accuracy')
plt.xticks(list(k))
plt.show()
```

Hasilnya:




```
# The best K with its score
print('Faktor K terbaik =', best_k)
print('Nilai akurasi =', score)
```

```
Hasilnya :

Faktor K terbaik = 9
Nilai akurasi = 0.9473684210526315
```

Hasil akurasi dengan metode KNN dengan menggunakan hanya menggunakan 2 features dan scaling sebesar 92.1%

e) Interpretasi

Dari plot di atas, dapat dilihat bahwa jumlah K terbaik adalah 9 dengan accuracy mencapai 94.7%. Semakin banyak K, trend training dan testing accuracy cenderung fluktuatif dan tidak lebih baik dari K = 9.

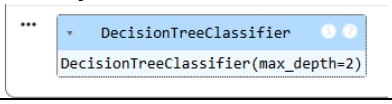
e. Decision tree

```
# Import Libraries
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.tree import plot_tree
```

```
# Define the model
tree = DecisionTreeClassifier(
    criterion='gini',
    max_depth=2
)
```

```
# Fitting
tree.fit(X_train, y_train)
```

Hasilnya :




```
# Test the model
y_predict_class = tree.predict(X_test)
```

```
# Metric result
print('Model accuracy in test dataset:',
accuracy_score(y_test, y_predict_class))
```

Hasilnya:

```
Model accuracy in test dataset: 0.8859649122807017
```

```
# proporsi kelas 0 (Benign) dan kelas 1 (Malignant)
y_train['diagnosis'].value_counts()
```

Hasilnya:

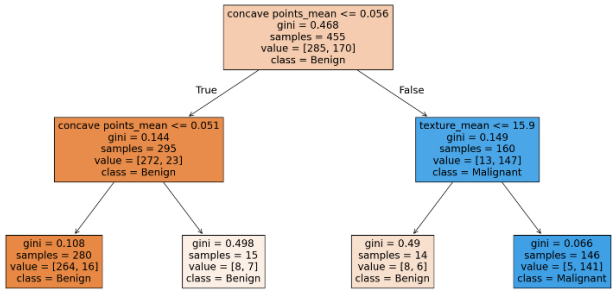
diagnosis	count
0	285
1	170

dtype: int64

```
# Tree plot
plt.figure(figsize=(20, 10))
```

```
plot_tree(tree, feature_names= list(X), class_names=
['Benign', 'Malignant'], filled=True);
```

Hasilnya:



a) With scaling

```
# Scaling
scaler = MinMaxScaler()
scaler.fit(X_train) # preprocess fit

X_train_scaled = scaler.transform(X_train) # transform
data X_train
X_test_scaled = scaler.transform(X_test) # transform data
X_test
```

```
# model
tree = DecisionTreeClassifier(
    criterion='gini',
    max_depth=2
)
```

```
# Fit
tree.fit(X_train_scaled, y_train)

# prdict
y_predict_class = tree.predict(X_test_scaled)
```

```
# Metric result
print('Model accuracy in test dataset with scaling:',
accuracy_score(y_test, y_predict_class))
```

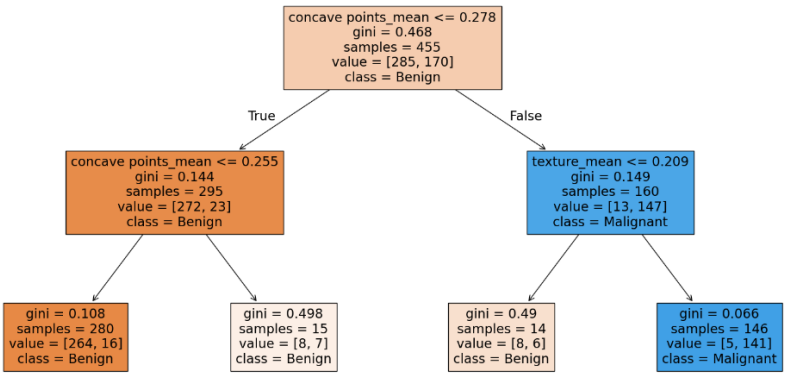
Hasilnya :

```
[177]
0s
Model accuracy in test dataset with scaling:
0.8859649122807017
```

```
# Tree plot
plt.figure(figsize=(20, 10))
```

```
plot_tree(tree, feature_names= list(X), class_names=
['Benign','Malignant'], filled=True);
```

Hasilnya:



b) Without max_depth

```
tree = DecisionTreeClassifier(
    criterion='gini'
)

tree.fit(X_train_scaled, y_train)
y_predict_class = tree.predict(X_test_scaled)
```

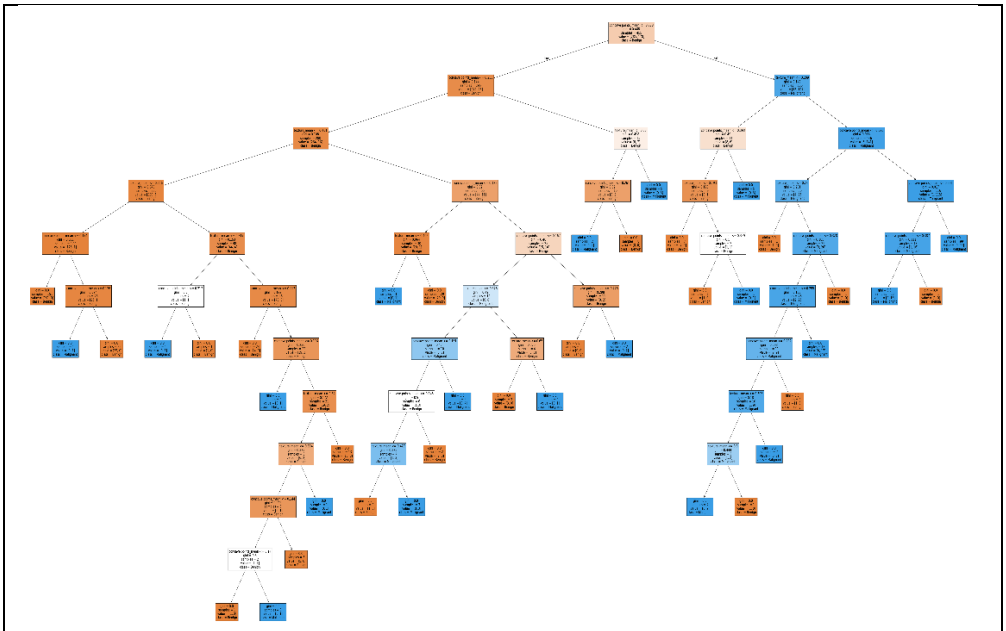
```
print('Model accuracy in test dataset:',
accuracy_score(y_test, y_predict_class))
```

Hasilnya :

```
Model accuracy in test dataset: 0.9035087719298246
```

```
plt.figure(figsize=(100, 50))
plot_tree(tree, feature_names= list(X), class_names=
['Benign', 'Malignant'], filled=True);
```

Hasil setelah di klik run :



```
y_train['diagnosis'].value_counts()
```

Hasilnya :

diagnosis	count
0	285
1	170

dtype: int64

c) Interpretasi

Dengan menggunakan model decision tree dengan depth=2, akurasi yang didapat adalah 88.5%. Lalu dilakukan scaling, namun hasil akurasi tidak berubah sama sekali. Hal ini terjadi karena meskipun dilakukan scaling, tidak ada perubahan pada heterogenitas data. Kemudian, dengan scaling, data menjadi sulit untuk di interpretasikan karena skalanya dipukul rata sehingga kita tidak mengetahui lagi, angka asli dari suatu kolom data. Dengan kata lain, model decision tidak membutuhkan scaling.

Dilakukan juga percobaan dengan tidak memasukkan jumlah maksimum depth. Dapat dilihat pada plot, proses otomatisasi melakukan klasifikasi hingga depth ke 11 dan nilai gini menjadi 0.00 semua pada node terakhir (sudah homogen). Hal ini menyebabkan adanya peningkatan akurasi menjadi 90%, tetapi model menjadi semakin sulit untuk diinterpretasi.

Dengan mengacu pada besarnya nilai akurasi, dapat disimpulkan bahwa model KNN dengan menggunakan scaling lebih baik daripada model Decision Tree untuk prediksi jenis kanker dengan hanya menggunakan 2 features.